

REGULAR ARTICLE

Systems Engineering With Architecture Modeling, Formal Verification, and Human Interactions for Learning-Enabled Autonomous Agent

Parth Ganeriwala¹  | Nandith Narayan¹ | Randolph M. Jones² | Michael Matessa³ | Siddhartha Bhattacharyya¹ | Jennifer Davis³ | Simone Fulvio Rollini³ | Hemant Purohit¹ | Natasha Neogi⁴

¹Florida Institute of Technology, Melbourne, USA | ²Soar Technology, LLC (SoarTech), Ann Arbor, USA | ³Collins Aerospace, Cedar Rapids, USA | ⁴NASA Langley Research Center, Hampton, USA

Correspondence: Parth Ganeriwala (pganeriwala2022@my.fit.edu)

Received: 29 January 2024 | **Revised:** 13 October 2024 | **Accepted:** 25 March 2025

Funding: This work was funded by award 80NSSC20M0005 to the NASA Langley Research Center from the NASA Shared Services Center.

Keywords: assuring learning enabled systems | autonomous agent | formal verification | human–autonomy interactions | model based systems engineering

ABSTRACT

With the rapid advancements in Artificial Intelligence (AI), autonomous agents are expected to handle increasingly complex situations. However, learning-enabled algorithms, which are critical to these systems, present significant challenges, including complexity, difficulty in verification, and a lack of certification pathways. A systematic approach integrating architectural analysis with human–machine interaction is crucial to ensuring the safety of these systems. This research emphasizes the early incorporation of human interactions in the design of architectural models to meet safety requirements. These interactions are modeled in the Soar cognitive architecture, which combines symbolic decision logic and numeric decision preferences, refined by reinforcement learning. The agent is then automatically translated into the formal verification environment, nuXmv, where its properties are verified. Our framework integrates systems modeling, formal verification, and simulation to check operational correctness, enhancing the reliability and safety of learning-enabled autonomous agents.

1 | Introduction

With the advancements in machine learning, it is expected that autonomous agents/systems will become increasingly intelligent. As a result, it is envisioned that autonomous agents will be capable enough to handle emerging situations. This is evident from the research explorations that are evaluating advanced capabilities such as autonomous collision avoidance [1, 2], advanced air mobility [3, 4], drone delivery systems [5, 6], and autonomous cars [7]. Campbell et al. [8] discuss the need for the integration of new advanced algorithms to ensure the safety of passengers and bystanders for autonomous Urban Air Mobility (UAM) [9, 10]. They also discuss how to assess the vehicle capability to handle

contingency scenarios. Gregory et al. [9] discuss the need for the use of deterministic and learning models to handle uncertain situations with the design of intelligent systems and the assessment of such vehicles. Although these advancements enhance the expertise of the autonomous system/agent, this makes assurance and certification more challenging. The integration of systems engineering with architecture modeling, formal verification, and consideration of human interactions play a crucial role in addressing the challenges of assurance and certification in the context of learning-enabled autonomous agents. Systems engineering enables a systematic approach to define requirements, conduct trade-off analyses, and discuss the design and structure of learning-enabled autonomous agents [11]. Formal

verification methods play a pivotal role in ensuring safety and handling uncertainty, offering a mathematically rigorous means of assessing system compliance with specified requirements [12]. Moreover, considerations of human interactions are paramount, emphasizing user-centric design, integration of user feedback, and ethical considerations in the design process [13].

In the research effort by [14–16], they discuss an architectural modeling approach that includes Human–Machine Interfaces (HMIs) and formal methods to verify the correctness of an Increasingly Autonomous System (IAS). For the assurance of learning-enabled IAS designed for contingency management, Neogi et al. [17] investigate chunking, a general learning mechanism in Soar [18]. They do not evaluate the need for human–machine interactions, and the learning mechanism is not based on machine learning using statistical inferences.

In the Assured Human–Machine Interface for Increasingly Autonomous Systems (AHMIAS) approach [14, 16], along with the integration of advanced technologies, we have identified that there is a need to discover the human–machine interaction that should be implemented for learning-enabled systems (LESS). In Section 2 related work is discussed, with the proposed methodology in Section 3. In Section 4, the human–machine interactions essential for our selected scenarios and the learning function of the IAS are identified and discussed. The design of and the experiments performed with, the learning-enabled IAS are explained in Section 5. Then, in Section 6, formal verification of the IAS is elaborated on, with the verification of learning-enabled autonomous systems. Finally, the results are in Section 7 with the conclusion in Section 8.

Our contribution is the development of a framework that integrates human–machine interactions and formal methods to ensure the safety of learning-enabled autonomous systems. We have focused on identifying key human–machine interactions necessary for collaboration between humans and machine learning-enabled agents. Specifically, we highlight the interaction where the system must manage a lack of human response, triggering an alert—critical for safe operation. Additionally, we extended our Soar-to-nuXmv translator to accommodate learning constructs within Soar. This allows us to formally verify the correctness and safety properties of the learning-enabled agent. The architecture models, IAS agent, translator code, and verified models are available on our project repository.¹

2 | Related Work

In the pursuit of increasing the learning capability of IAS, various methodologies, including deep learning, markov decision processes, data analysis, and real-time discoveries have been proposed to achieve significant enhancements in cognitive autonomy [19–21]. On the other hand, within the controls research community, constrained control approaches such as Model Predictive Control (MPC) ensure the learning based on the previously recorded data. MPC has gained the researchers' attention on the basis of the methods being used in learning that include learning through system dynamics, learning the controller design, and MPC for safe learning [22–24]. LESSs have also been designed on the basis of goal-based, model-driven

approaches to assure self-adaptation and self-assessment [25–28]. Additionally, various uncertainties about the safety assurance of LESSs have been exhibited in Model-Driven Assurance for Learning-Enabled Autonomous Systems (MoDALAS) [29].

Formal methods have been consistently used in assuring safety and trust in safety critical applications, intelligent systems, and co-operative autonomous agents [15]. Cofer et al. [30] discuss a formalized run-time assurance architecture together with diverse monitors that were used to ensure the safety of an aircraft taxiing application that included a neural-network-based learning-enabled component. Moreover, the research conducted by Bhattacharyya et al. [14] shows that the human–machine team architecture can be designed and verified using Architecture Analysis and Design Language (AADL) [31, 32] and the Assume Guarantee REasoning Environment (AGREE) [33], which lays a strong foundation in increasing trust and safety. To ensure the trustworthiness and safety of Learning-Enabled IAS (LEIAS), the extension of formal verification techniques becomes paramount [12, 34]. Systems engineering methodologies [35, 36], combined with architecture modeling [37], can offer a structured approach to define requirements, conduct trade-off analyses with automated reasoning, and articulate the design and structure of LEIAS [38, 39]. Incorporating human interactions, particularly in the learning phase, is essential for user-centric design and the iterative improvement of the LEIAS's performance based on user feedback. This paper extends [14, 16] by the addition of learning to the IAS to create a LEIAS, updating the architecture models, extending the Soar-to-nuXmv translator to support learning, and verifying properties related to outcomes of learning.

3 | Research Methodology

The proposed methodology for the design of LEIAS is shown in Figure 1. During the Requirements Phase, the scenarios for learning are developed, along with the specifications for the HMIs. These are then mapped into the architectural components that are required for learning to occur. Then, the behavior of the components are added in the Implementation Phase, as learning infrastructure, the design of the learning agent and, then, experiments are performed for the LEIAS to learn the specified behavior. Finally, during the Testing and Verification phase, formal verification of the LEIAS is performed by extending the previous automated translator with the inclusion of the translation of learning-related constructs.

4 | HMI and Architecture of LEIAS

4.1 | Human–Machine Interface (HMI)

Scenarios are developed to determine the roles and responsibilities of a human pilot working together with an IAS to enable UAM operations. In the unreliable sensor scenario, the IAS monitors and determines the aircraft position from a combination of GPS, LiDAR, and IMU sensors. After entering an urban canyon, the reliability of GPS for determining location is reduced, leaving LiDAR and IMU reliable. The IAS detects difference between the GPS position value and LiDAR and IMU values, and alerts that the GPS is unreliable, asking the pilot for agreement. The

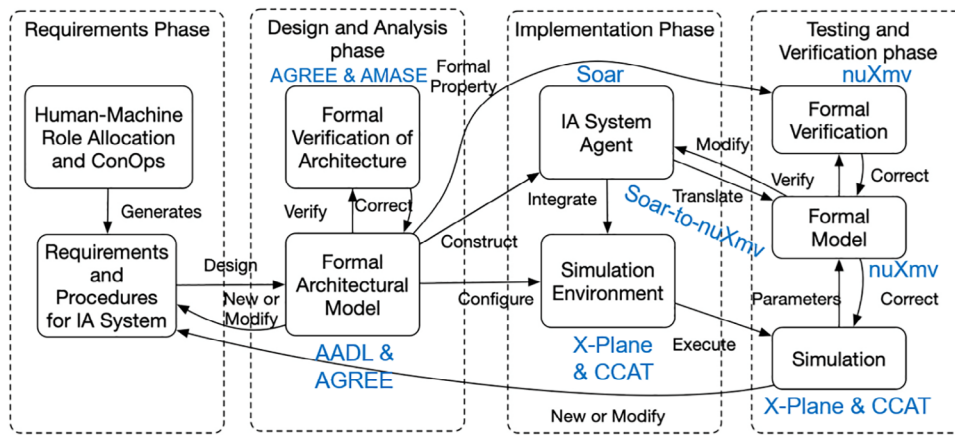


FIGURE 1 | Proposed methodology for LEIAS.

pilot then agrees or disagrees with the unreliable GPS sensor assessment, and the IAS learns the pilot's preference for alerting. To extend the unreliable sensor scenario with learning, an HMI is needed to enable the interaction between the IAS and the pilot, to allow the IAS to learn from the pilot's feedback.

The HMI display consists of a map, position indicators (for GPS, LiDAR, and IMU sensors), and a triangular unreliable sensor alerting icon placed next to a severity indicator bar. The position indicators show the pilot any discrepancy between sensor positions in units that a pilot can understand (e.g., within a building, within a block, or across town).

The error range is decomposed into four segments: a normal range for minimal sensor discrepancy, a level 1 range for larger discrepancy, a level 2 range for even larger discrepancy, and a safety range for a significant sensor discrepancy. The IAS always provides an alert for the safety range, never provides an alert for the normal range, and learns the pilot's preference for level 1 and level 2; it colors the corresponding bar segment red if an alert is to be given and green if an alert is not to be given.

4.2 | Human-Machine Team Architecture Model

We capture the structure of the model (components and interfaces) in AADL. AADL is a standardized language for embedded, real-time systems. It supports design, analysis, virtual integration, and code generation. We include in the model the human pilot, the IAS, and three position sensors. We capture an abstract representation of the behavior of the system and each subcomponent in an assume-guarantee contract in AGREE. AGREE performs compositional analysis, verifying system requirements based on the composition of the component assume-guarantee contracts. By abstracting the implementation of subsystems and software components into formal contracts, large systems can be verified hierarchically without the need to perform a monolithic analysis of the entire system at once. AGREE translates the model to the Lustre language and then performs verification using a model checker (e.g., JKind [40]) and an SMT solver (e.g., Z3 [41]).

The structure of the human-machine team architecture model builds on the AHMIAS framework introduced by Bhattacharyya

et al. [14], where two scenarios are distinguished, one characterized by a flawless interaction, and one where the IAS is subject to a fault or the pilot acts in an unanticipated manner. Verification of system requirements in the latter scenario is performed with AMASE [42], a tool that extends AGREE with safety analysis capabilities, relying on a safety annex that allows to reason about faults and faulty components behaviors.

Nominal Behavior. In the model, at each step two sensors, not necessarily distinct, are respectively marked as “active”, if currently in use, and “recommended”, if suggested for the next step, for example, to replace the active sensor in case of unreliability. The IAS interface informs on the active and recommended sensors, and is extended to allow the communication of the discrepancy value, the corresponding range, and an unreliability alert for the active sensor; the pilot's interface is extended to provide an acceptance/rejection feedback to an IAS alert. Requirements are added in terms of guarantees in the AGREE language, to specify the IAS behavior and to express expectations on the pilot's behavior. For example, the IAS (i) should not evaluate a sensor as unreliable if the sensor's error range falls into the Normal range. This is implemented in the guarantee *ias_g1*, which ensures that for each of *sensor1*, *sensor2*, and *sensor3*, if their respective *sensorErrorRange* is equal to *enum(Types_Constants::SensorErrorRange, Normal)*, then the corresponding *sensorReliable* flag is set to true. And conversely, (ii) it should determine unreliability if the sensor's error range falls into the *Safety range*. This is captured in the guarantee *ias_g2*, which specifies that for each sensor, if its *sensorErrorRange* is equal to *enum(Types_Constants::SensorErrorRange, Safety)*, then the respective *sensorReliable* flag is set to false (implemented as not *sensorReliable*). These guarantees (AADL-AGREE Contract 1) ensure that the IAS correctly evaluates sensor reliability based on their error ranges, maintaining consistency between the Normal range and reliability, and between the Safety range and unreliability for all three sensors in the system.

At the system level, requirements on the interaction between the IAS and the pilot are encoded as AGREE lemmata and subject to formal verification.

Whenever a sensor discrepancy falls into level 1 or level 2 range, the learning process is activated. Broadly speaking, after the pilot

```

guarantee ias_g1 "If a sensor error is in Normal range, then
    the sensor is evaluated as reliable":
    ((sensor1ErrorRange =
        enum(Types_Constants::SensorErrorRange, Normal))
    => sensor1Reliable)
and ((sensor2ErrorRange =
    enum(Types_Constants::SensorErrorRange, Normal))
    => sensor2Reliable)
and ((sensor3ErrorRange =
    enum(Types_Constants::SensorErrorRange, Normal))
    => sensor3Reliable);

```

AADL-AGREE CONTRACT 1 | IAS reliability evaluation.

```

guarantee h_g1 "If an active sensor unreliability alert is
    received by the operator, then a response is
    communicated by the operator at the next step":
(not previouslyActiveSensorReliable)
<=>
(
    (Output.activeSensorUnreliableResponse =
        enum(Types_Constants::Response, Agree))
    or
    (Output.activeSensorUnreliableResponse =
        enum(Types_Constants::Response, Disagree)));

```

AADL-AGREE CONTRACT 2 | Pilot's response to alert.

responds to an alert in the presence of an active sensor discrepancy, the system's behavior is adjusted based on this feedback. If the pilot's response is *Agree* (acceptance), it should become more "likely" for the IAS to issue an alert for that sensor and discrepancy in the future. Conversely, if the response is *Disagree* (rejection), it should become less "likely" for the IAS to issue such an alert. The precise definition of "likelihood" and the specifics of how it is affected by the pilot's response depend on the particular learning algorithm and implementation; for these reasons, they are not represented at the architecture level. However, the IAS-pilot communication flow, which enables this learning process, is modeled through guarantees on both the IAS and the pilot's side. For example, the pilot is expected to provide feedback to an alert from the IAS within a given amount of time, as represented in AADL-AGREE Contract 2. Specifically, guarantee *h_g1* ensures that if an active sensor unreliability alert is received by the operator (indicated by *not previouslyActiveSensorReliable*), then a response is communicated by the operator at the next step. This response can be either *Agree* or *Disagree*, as represented by *Output.activeSensorUnreliableResponse* being set to the corresponding enumeration value from *Types_Constants::Response*. Conversely, guarantee *h_g2* specifies that if no active sensor unreliability alert is received by the operator (indicated by *previouslyActiveSensorReliable*), then no response is communicated by the operator at the next step. In this case, *Output.activeSensorUnreliableResponse* is set to *Neutral*. These guarantees ensure a consistent communication pattern between the IAS and the pilot, providing a framework for the learning process to occur based on the pilot's feedback to sensor unreliability alerts.

Unexpected Behavior. As part of the safety analysis, an unanticipated behavior is introduced on the pilot's side (AADL-AGREE Contract 3) in the form of a lack of response to an IAS unreliability alert. This is implemented through the *fail active sensor unreliability response* node, which models a scenario where the pilot fails to provide the expected feedback. The node takes two inputs; *h2i_in* of type *H2I.impl*, representing the normal input from the pilot to

```

guarantee ias_g2 "If a sensor error is in Safety range, then
    the sensor is evaluated as unreliable":
    ((sensor1ErrorRange =
        enum(Types_Constants::SensorErrorRange, Safety))
    => (not sensor1Reliable))
and ((sensor2ErrorRange =
    enum(Types_Constants::SensorErrorRange, Safety))
    => (not sensor2Reliable))
and ((sensor3ErrorRange =
    enum(Types_Constants::SensorErrorRange, Safety))
    => (not sensor3Reliable));

```

```

guarantee h_g2 "If no active sensor unreliability alert is
    received by the operator, then no response is
    communicated by the operator at the next step":
previouslyActiveSensorReliable
<=>
Output.activeSensorUnreliableResponse =
enum(Types_Constants::Response, Neutral);

```

```

node fail_active_sensor_unreliability_response(h2i_in: H2I.impl, trigger: bool)
    returns (h2i_out: H2I.impl);
let
    h2i_out = if (trigger)
    then h2i_in{activeSensorUnreliableResponse := enum(Response, Neutral)}
    else h2i_in;
tel;

```

AADL-AGREE CONTRACT 3 | Pilot's lack of response.

the IAS, and *trigger* of type *bool*, which activates the fault when set to true. The node returns *h2i_out*, also of type *H2I.impl*, which represents the potentially modified output. When the *trigger* is true, the node overrides the *activeSensorUnreliableResponse* field of the input, setting it to *enum(Response, Neutral)*. This simulates a scenario where the pilot gives no response (*Neutral*) even when an alert is active, contrary to the expected behavior defined in the guarantees. The rationale behind this fault model is to challenge the system when the assumption that the pilot always gives feedback after being alerted of active sensor unreliability fails to hold. This allows for testing the robustness of the IAS and the overall system in scenarios where expected human-machine interaction patterns are not followed, potentially revealing safety-critical issues or areas for improvement in the system's design.

Enabling the unanticipated behavior (pilot's lack of response) leads to the violation of two system-level requirements (Figure 2). The first requirement, which encodes the expectation of a pilot's response, trivially does not hold when the response is absent. This requirement should be reconsidered if the original assumptions about the pilot's behavior are modified.

The second requirement, defining the conditions under which an unreliable sensor is acceptable as the active sensor, is shown in AADL-AGREE Contract 4. This requirement, expressed as lemma *t_l42*, states that if an unreliable sensor is the active sensor, one of three conditions must be true. The pilot disagreed with the IAS assessment (*activeSensorUnreliableRe-*

✓ lemma: [t_J20] If the active sensor error is in Normal range, then no active sensor unreliability alert is issued	Valid (1s)
❗ lemma: [t_J35] Operator responds to sensor unreliability alerts	Invalid (1s)
✓ lemma: [t_J39] Unless the active sensor becomes unreliable,	Valid (1s)
✓ lemma: [t_J40] If the operator agrees that the active sensor is unreliable,	Valid (1s)
✓ lemma: [t_J41] If the operator disagrees that the active sensor is unreliable, then the active sensor should not change.	Valid (1s)
❗ lemma: [t_J42] If an unreliable sensor is the active sensor, it must be the case that either	Invalid (1s)
✓ lemma: [t_J48] Sensor errors are always non-negative	Valid (1s)
✓ lemma: [t_J44] Active sensor in range	Valid (1s)
✓ lemma: [t_J45] Recommended active sensor in range	Valid (1s)
✓ lemma: [t_J46] Only stated transitions are allowed.	Valid (1s)

FIGURE 2 | Pilot's lack of response–verification results.

```

lemma t_l42 "If an unreliable sensor is the active sensor, it
must be the case that either the pilot disagreed
with the IAS assessment or the sensor just became
unreliable on this timestep or there was no
reliable sensor available on the previous timestep":
(
  ((IAS.Output.Active_Sensor = 1)
  and (not IAS.Output.activeSensorErrorData.reliable))
=>
(
  (Human.Output.activeSensorUnreliableResponse =
    enum(Response, Disagree))
  or prev(IAS.Aux.sensor1Reliable, true)
  or (not previouslyReliableSensorAvailable)))

```

```

and (
  ((IAS.Output.Active_Sensor = 2)
  and (not IAS.Output.activeSensorErrorData.reliable))
=>
(
  (Human.Output.activeSensorUnreliableResponse =
    enum(Response, Disagree))
  or prev(IAS.Aux.sensor2Reliable, true)
  or (not previouslyReliableSensorAvailable)))
and (
  ((IAS.Output.Active_Sensor = 3)
  and (not IAS.Output.activeSensorErrorData.reliable))
=>
(
  (Human.Output.activeSensorUnreliableResponse =
    enum(Response, Disagree))
  or prev(IAS.Aux.sensor3Reliable, true)
  or (not previouslyReliableSensorAvailable)))

```

AADL-AGREE CONTRACT 4 | Pilot's lack of response - violated requirement.

Step	0	1
lemma: [t_J42] If an unreliable sensor is the active sensor, it must be the case that either the pilot disagreed with the IAS assessment or the sensor just became unreliable on this timestep or there was no reliable sensor available on the previous timestep	TRUE	FALSE
The operator does not communicate a response	FALSE	TRUE
previouslyReliableSensorAvailable	TRUE	TRUE
Human		
Human.Output.activeSensorUnreliableResponse	Response_Neutral	Response_Neutral
IAS		
IAS.Aux.sensor1Reliable	FALSE	FALSE
IAS.Output.Active_Sensor	1	1
IAS.Output.activeSensorErrorData.errorRange	SensorErrorRange_Level1	SensorErrorRange_Level2
IAS.Output.activeSensorErrorData.reliable	FALSE	FALSE
IAS.previousActiveSensor	1	1
IAS.previousActiveSensorReliable	TRUE	FALSE
IAS.previousRecommendedSensor	1	2

FIGURE 3 | Pilot's lack of response–counterexample.

sponse = enum(Response, Disagree)), the sensor just became unreliable on this timestep (*prev(IAS.Aux.sensorXReliable, true)*, where *X* is 1, 2, or 3 depending on the active sensor), or there was no reliable sensor available on the previous timestep (*not previouslyReliableSensorAvailable*). The two-step counterexample generated by AMASE (Figure 3) corresponds to a situation where the active sensor (*IAS.Output.Active_Sensor*) is evaluated as unreliable (*not activeSensorErrorData.reliable*) at both steps, but the pilot's lack of feedback means that *Human.Output.activeSensorUnreliableResponse* is neither *Agree* nor *Disagree*. Furthermore, there is a reliable sensor available at the first step, which could have been chosen to replace the active sensor at the second step. This violates the third condition of the lemma, as *previouslyReliableSensorAvailable* would be true. The values at the first step that refer to a previous step are conventionally set, but the violation occurs because the system maintains an unreliable active sensor despite the availability of a reliable alternative, and without the expected disagreement from the pilot. This counterexample highlights the importance of the pilot's feedback in the system's decision-making process and reveals a potential vulnerability when this expected interaction is absent.

The analysis of the Human–Machine Team Architecture Model reveals the importance of robust system design in handling unanticipated human behaviors. By refining the IAS rule for active sensor determination (as shown in AADL-AGREE Contract 5), we can make the system more resilient to scenarios such as the pilot's lack of response. This refinement, verified using AMASE, restores the validity of the second requirement lemma (Figure 4). Furthermore, the degree of autonomy granted to the IAS is a crucial consideration. AADL-AGREE Contract 6 presents an alternative refinement that allows the IAS to act independently only when the active sensor discrepancy is in the safety range, potentially risking the pilot's life. This approach strikes a balance between system autonomy and human oversight.

With the human and system modeling analysis completed in AGREE, the next step involves representing these interactions in the Soar-Xplane simulation environment. This transition from formal modeling to simulation provides a platform for more dynamic testing and validation of the human–machine interactions.

5 | Learning Enabled Increasingly Autonomous System Design

The design of a learning-enabled IAS in this research effort involves using and evaluating learning capabilities in the cognitive architecture Soar. We extend the formal analysis conducted in the Human–Machine Team Architecture Model and, focus on the implementation of a Learning-Enabled Increasingly Autonomous System (LEIAS). The utilization of Soar's cognitive architecture enables the exploration of adaptive decision-making processes, potentially augmenting the system's capacity to navigate complex scenarios and respond effectively to

```

guarantee ias_g20 "The active sensor should stay the same unless
    (i) it is unreliable,
    (ii) the operator agrees or (the operator does not respond after one step and the active sensor error is in the Safety range), and
    (iii) there is another reliable sensor available":

Output.Active_Sensor =
if (previouslyRecommendedSensor = NIL)
then previousActiveSensor
else if (((not previouslyActiveSensorReliable) and ((InputFromHuman,activeSensorUnreliableResponse = enum(Response, Agree))
or ((InputFromHuman,activeSensorUnreliableResponse = enum(Response, Neutral))
and (previouslyActiveSensorErrorRange = enum(Types_Constants::SensorErrorRange, Safety))))))
then previouslyRecommendedSensor
else previousActiveSensor;

```

AADL-AGREE CONTRACT 5 | IAS active sensor determination - refinement.

✓ lemma: [t _{J20}] If the active sensor error is in Normal range, then no active sensor unreliability alert is issued	Valid (1s)
✗ lemma: [t _{J35}] Operator responds to sensor unreliability alerts	Invalid (1s)
✓ lemma: [t _{J39}] Unless the active sensor becomes unreliable,	Valid (1s)
✓ lemma: [t _{J40}] If the operator agrees that the active sensor is unreliable,	Valid (1s)
✓ lemma: [t _{J41}] If the operator disagrees that the active sensor is unreliable, then the active sensor should not change.	Valid (1s)
✓ lemma: [t _{J42}] If an unreliable sensor is the active sensor, it must be the case that either	Valid (1s)
✓ lemma: [t _{J48}] Sensor errors are always non-negative	Valid (1s)
✓ lemma: [t _{J44}] Active sensor in range	Valid (1s)
✓ lemma: [t _{J45}] Recommended active sensor in range	Valid (1s)
✓ lemma: [t _{J46}] Only stated transitions are allowed.	Valid (1s)

FIGURE 4 | Pilot's lack of response–verification results after IAS refinement.

```

guarantee ias_g20 "The active sensor should stay the same unless
    (i) it is unreliable,
    (ii) the operator agrees or (the operator does not respond after one step and the active sensor error is in the Safety range), and
    (iii) there is another reliable sensor available":

Output.Active_Sensor =
if (previouslyRecommendedSensor = NIL)
then previousActiveSensor
else if (((not previouslyActiveSensorReliable) and ((InputFromHuman,activeSensorUnreliableResponse = enum(Response, Agree))
or ((InputFromHuman,activeSensorUnreliableResponse = enum(Response, Neutral))
and (previouslyActiveSensorErrorRange = enum(Types_Constants::SensorErrorRange, Safety))))))
then previouslyRecommendedSensor
else previousActiveSensor;

```

AADL-AGREE CONTRACT 6 | IAS active sensor determination - refinement alternative.

unanticipated events. This approach aims to synthesize the formal guarantees established in the previous section with dynamic learning mechanisms, thereby advancing the system's autonomous capabilities while maintaining safety and reliability standards.

5.1 | Learning in the Soar Cognitive Architecture

The problem of V&V for intelligent systems is further complicated when those systems are able to learn and improve their behavior with experience [13]. As our case study focuses in part on an IAS implemented in the Soar cognitive architecture [18], here we describe reinforcement learning, as that is what was integrated for this study. Soar supports other forms of learning, that is, chunking, episodic learning, and semantic learning [18] (Figure 5).

Reinforcement learning is a well established form of learning in the machine learning research community [43]. Soar's particular implementation of reinforcement learning integrates the learning algorithms with Soar's native decision-making structure [44]. In Soar, decisions are made by observing the current state of the

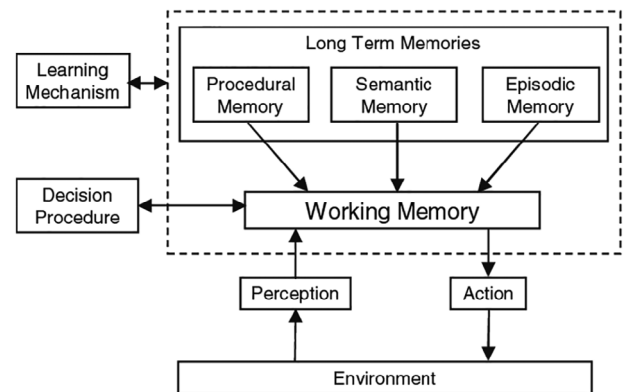


FIGURE 5 | Soar cognitive architecture process flow.

world, suggesting candidate inferences or actions (“operators”) to execute next, determining which of those candidates should be most preferred in the current context, and then selecting a single inference or action to execute. This decision cycle repeats quickly and indefinitely to implement a Soar agent’s intelligent reasoning and behavior.

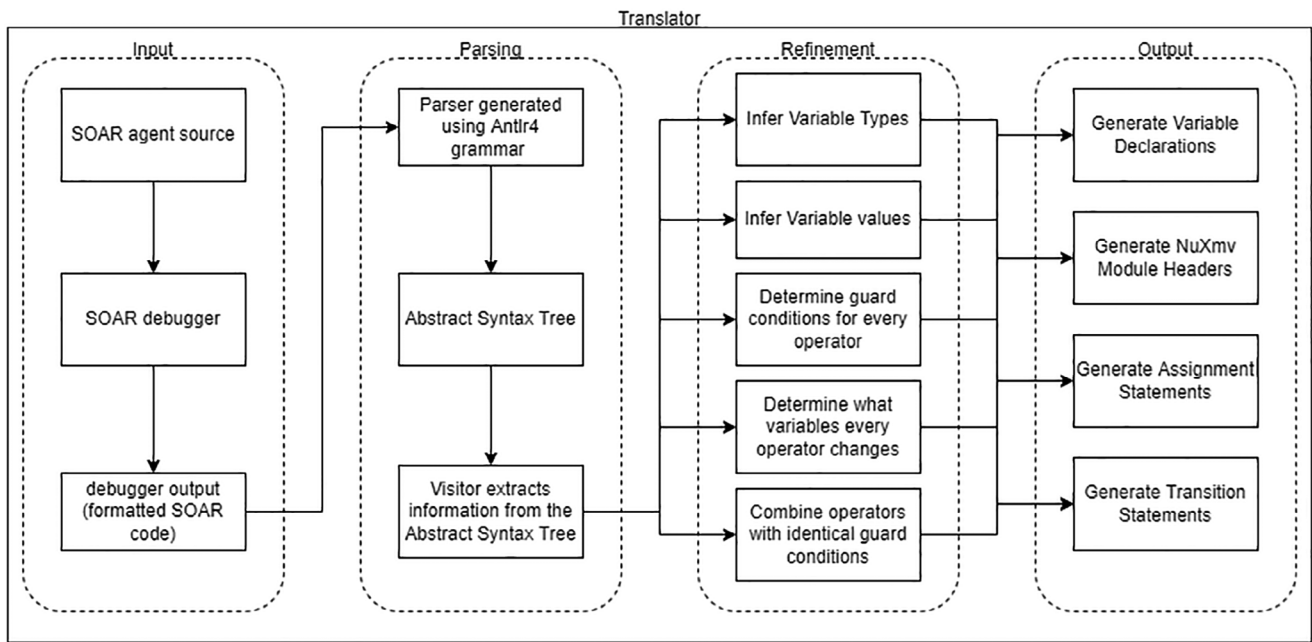


FIGURE 6 | Translation from LEIAS in Soar to nuXmv.

5.2 | Design of the Integrated Learning Agent

Our research effort initially focused on developing a Soar decision-making agent with no learning [14]. This agent makes a relatively simple set of decisions about how to control the aircraft during different phases of flight, including takeoff, transit, and landing, as well as transitions between these phases. The agent also includes some knowledge about how to address situations in which it infers that one of its position sensors is faulty. This includes decision-making logic about alerting a human pilot and using the pilot's guidance about whether to decide that a sensor is unreliable. The use case focuses on the design of an intelligent IAS that learns the human pilot's preferences about the conditions in which the pilot wants to be alerted about a potential sensor failure.

Symbolic Decision Logic-Based Learning Agent. The LEIAS agent is structured based on regions of alerting levels as described in Section IV B. For normal range of error, the system will never alert the pilot. For safety range of error, the system will always alert the pilot. For errors in level 1 or 2, initially, the Soar agent decides to generate an alert. Over time, the system is given a reward when it alerts the pilot and the pilot "accepts" the alert, and a punishment when the pilot "rejects" the alert. With each new experience, the reinforcement learning algorithm incrementally allocates portions of these reward/punishment signals to the decisions.

5.3 | Experiments for LEIAS

To conduct the experiments, the overall learning architecture included the integration of the learning infrastructure, the interactions with the environment, and the symbolic decision logic-based learning agent. Cycles of training and testing were executed to validate that learning.

The **learning infrastructure** consisted of all the necessary entities and interactions that support the generation of data. One of the infrastructural requirements was the need to store the data, as reinforcement learning requires several cycles to process it. The second requirement was to store the threshold values and the third requirement was to store the pilot's response. To expedite data collection, the pilot's response was scripted as part of the infrastructure.

5.4 | Experimental Setup and Results for LEIAS

The learning process was split into a series of learning trials and testing trials. During each learning trial, a random error is introduced and the IAS decides whether to warn the pilot. If the IAS decision matches the preferences of the scripted pilot, then the agent is rewarded with a value of +1. If the IAS decision does not match the scripted pilot, then the agent is given a reward value of -1. In our scenario, the scripted pilot wants the IAS agent to always warn when the error is greater than or equal to 9 (level 2). During each testing trial, the introduced sensor error continuously increases over time until it reaches safety error range. These trials validated that the agent's learned behavior. Boltzmann with high and low temperatures [45] and simulated annealing [46] were explored as the rule selection policy.

Results. The experiments applying the Boltzmann policy with a high temperature of 25 yielded significant insights into the LEIAS's learning behavior. When analyzing the pairwise error values of the inputs received, we observed distinct patterns in the system's response to different error levels. For error values in level 2 (generally higher error values), the LEIAS consistently received positive reinforcement for issuing warnings. This led to the system learning to reliably warn in high-error scenarios. Conversely, for error values in level 1 (specifically between 8 and 9), the LEIAS received negative reinforcement for issuing

ALGORITHM 1 | Generate Infinite State Machine $ISM = (S, S_0, Vars, G, Act, Tr)$ from $rname(V, (pre\{\phi_1, \phi_2, \dots, \phi_m\}, post\{\psi_1, \psi_2, \dots, \psi_r\}))$.

```

1: Initialize  $ISM_i = (S \leftarrow \{Start, Run\}, s_0 \leftarrow \{Start\},$ 
    $Vars = \{state\_superstate = nil\}, G = \{\}, Act = \{\}, Tr = \emptyset,$ 
    $Ebr = \emptyset)$ 
2: Output  $ISM = (S, S_0, Vars, G, Act, Tr)$ 
3: MODULE: VAR state: {start, run};
4: next (state) = Case
5: for all  $i, j \in \{1, \dots, m\}, pre\{\phi_i\}, post\{\phi_i\}$  do
6:    $Tr \leftarrow (state = start \ \& \ G_i == pre(\phi_i) : run);$ 
7:    $Tr \leftarrow (state = start \ \& \ G_i == post(\psi_i) : run);$ 
8:    $Ebr \leftarrow (state = start \ \& \ G_j == pre(\phi_j) : run);$ 
9:    $Ebr \leftarrow (state = start \ \& \ G_j == post(\psi_j) : run);$ 
10: end for
11: for all  $i \in \{1, \dots, m\}, state\_operator\_name \in pre\{\phi_i\}$  do
12:   if {state\_operator\_name count == 1} then
13:     if {rulei : rulei == rulej} then
14:       assign value rulei  $\leftarrow \{r_i\}$ ;
15:       assign impasse rule  $r_{combine}$  to list  $r_i \leftarrow \{r_i \dots r_j\}$ ;
16:       remove  $r_{combine}$ ;
17:     else if { $r_i == r_j$ } then
18:       remove  $r_j$ ;
19:       Print TRANS
20:     end if
21:   end if
22:    $Tr \leftarrow (next(state\_operator\_name) = \{state ==$ 
    $run \ \& \ G_i == pre(\phi_i)\} : v_i)$ 
23:    $Ebr \leftarrow (next\{r_i \in elaborate \ \& \ G_j == pre(\phi_i)\} : v_i)$ 
24:   if  $v_{i\_priority} > v_{j\_priority}$  then
25:     Generate the  $next(state\_operator\_name)$ 
     statement on top;
26:   else
27:     Generate the  $next(state\_operator\_name)$ 
     statement on the bottom;
28:   end if
29: end for
30: for all  $i \in \{1, \dots, r\}, Vars \in post\{\phi_i\}$  do
31:   if {vari count == 1} then
32:     Print TRANS
33:   end if
34:    $Tr \leftarrow (next(var_i) = state = run \ \& \ G_i = (\psi_i) : v_i)$ 
35:    $Ebr \leftarrow (next(var_j) = r_i \in elaborate \ \& \ G_j = (\psi_j) : v_j)$ 
36:    $Act \leftarrow Act \ \wedge \ (next(var_i) = v_i)$ 
37: end for

```

warnings. As a result, the system learned to refrain from warning when the error fell below 9. Similar behavioral patterns were observed in experiments using the Boltzmann policy with low temperature and in those employing simulated annealing. However, it is noteworthy that the Boltzmann policy with high temperature exhibited more extensive exploration of the search space compared to the other algorithms. This increased exploration manifested in a more gradual learning process, aligning with theoretical expectations for high-temperature Boltzmann exploration. These results demonstrate the LEIAS's ability to adapt its warning behavior based on the magnitude of input errors, showcasing the effectiveness of the implemented learning mechanisms across different policy configurations.

6 | Verification of LEIAS

The verification process involved translation from Soar to nuXmv and then performing verification by means of the nuXmv tool. This includes sophisticated SMT-based model-checking techniques and expands the NuSMV language with additional data types, such as integers and reals, for infinite-state systems [47].

6.1 | Translation from Soar to nuXmv

The translation from the LEIAS built-in Soar to nuXmv involves four phases: Input, Parsing, Refinement, and Output (Figure 6). During the Input phase, the Soar agent source file is transformed into a runtime execution file, by the Soar debugger. During the Parsing phase, the ANTLR [48] grammar representing the Soar grammar rules is used to create an abstract syntax tree, which is in turn used to extract relevant information. During the Refinement phase, the variable types and values are identified, followed by combining the pre and post-conditions associated with each of the Soar rules. Finally, during the output phase, a nuXmv file is generated that consists of the variables, modules, transition, and guard conditions, as is expected for a state machine representation of the LEIAS translated from Soar. The algorithm for the translator is shown in Algorithm 1.

To handle constructs associated with learning in Soar, the changes enumerated below were made to the translator:

- **Priority With Preference Values:** When using reinforcement learning, Soar prioritizes rules by assigning preference values, however, in nuXmv, priority is executed in a top-down order. The preferred order is computed and then generated by the translator (Lines 47–52). We define a Soar production rule as a function of a finite set of variables $v_i, v_j \in V$, where $i, j = 1, 2, 3, \dots, n$, whose valuation $val(V) = v_i, v_j$ represent the state of the system. While generating the state operator name, the satisfaction of the precondition is checked along with any priorities associated with the value of the state operator. The preference values are compared and the rules are generated in the order from highest to lowest priority.
- **Elaboration Rules:** The elaboration phase is when rules, that elaborate the scenario, propose operators, and selected operators are fired. The numeric mapping of symbolic constants used for learning a range of values in Soar was performed

TABLE 1 | Specifications for LEIAS behavior.

Learning specification in English	formal specification
If the sensor error is at or past the safety threshold, the agent shall issue a warning.	LTLSPEC F (X state_imu-error-info_warn-condition = safety & state_operator_name = warn)
If error difference within sensors is non zero, then the agent shall execute the error changed operator	LTLSPEC F (X state_imu-error-info_current-value != state_imu-error-info_old-value & state_operator_name = record-changed-error-imu)
If the difference in error values hasn't changed, the agent shall not execute the error changed operator	LTLSPEC G (X state_operator_name = record-changed-error-imu → state_imu-error-info_value-changed = nil & (state_imu-error-info_current-value = state_imu-error-info_old-value))
When a sensor is at fault, and the warning level is low, the agent shall not warn	LTLSPEC F (X state_imu-error-info_warn-condition = low & state_operator_name = warn)
When a sensor is at fault, and the warning level is high, the agent shall warn	LTLSPEC F (X state_imu-error-info_warn-condition = high & state_operator_name = warn)
If the sensor error is within normal threshold, the agent shall not issue a warning	LTLSPEC F (X state_imu-error-info_warn-condition = normal & state_operator_name = do-not-warn)
If a warning has previously been issued, the agent shall not issue another warning	LTLSPEC F (X (state_warning-issued != nil & state_imu-error-info_warn-condition = safety) → (state_operator_name = do-not-warn))
The agent shall resolve an impasse in selecting learned rules	LTLSPEC F (X (state_imu-error-info_warn-condition = high) → state_operator_name = combined-warn-propose-do-not-warn-propose))
The agent shall always follow the preference order from the learned rules	LTLSPEC F (X state_imu-error-info_warn-condition = normal → state_operator_name = do-not-warn)

with the use of elaboration rules. The translation handled this mapping of symbolic constants to numeric values for the LEIAS. Elaboration rules are fired at the start of each SOAR cycle. We define a Soar elaboration rule (Lines 31–58) as a function of a finite set of variables $v_j \in V$, where $j = 1, 2, 3 \dots m$, along with a finite set of well-formed formulae (WFF) $\phi = \{\phi_1, \phi_2, \dots \phi_m\}$, representing the left-hand side of the Soar production rule (e.g., the preconditions), and a finite set of WFF $\psi = \{\psi_1, \psi_2, \dots \psi_r\}$, representing the actions embodied by the right-hand side of the Soar elaboration rule.

- **Existence Checks:** In Soar, the existence of a variable is checked, before the evaluation of any condition associated with the variable. As part of the learning process, some values needed to be removed before new values were assigned. As a result, the existence check was also translated for integers and real values (Lines 3–21). For example, some of the variables in nuXmv executing existence are *state_gps-error-info_current-value_exists* = yes, *state_safety-error_exists* = no.
- **Handling of Impasse Rules:** During the *propose* cycle, Soar identifies the set of rules for which the conditions match, then, during the *apply* cycle, it checks the preference values of the rules and selects the one that has the highest preference value. During the translation to nuXmv, the proposed rules were combined together (Lines 34–44); without the merging of these rules, nuXmv would have always selected the first apply operator that was generated. For example, the following nuXmv translation, (*state* = run & *state_ready-to-decide* = yes): *combined-warn-propose-do-not-warn-propose*; translated the

condition of whether the agent has set the ready-to-decide flag to yes and then sets the state_operator_name to combined-warn-propose-do-not-warn-propose which handles the impasse of proposing warn or do-not-warn operators. Then the translator generated the apply rules based on preference values in a top-down order from highest to lowest preference value, as nuXmv selects the one that appeared first when the condition of more than one rule matches.

6.2 | LEIAS Properties and Verification in nuXmv

The formal verification of the learning-enabled agent was performed by decomposing the rules of the LEIAS into two categories, one that was generated based on learning, and the other that focused on the non-learning aspects. The LEIAS agent was trained to learn the threshold of the pairwise sensor (GPS, IMU, LiDAR) error difference at which the pilot is warned, or not warned. The preference values were recorded as learning occurred. Then the rules were translated into nuXmv. An input template was created in nuXmv for the generation of the input values that the IAS agent responds to. These input values updated the learning parameters, along with the change of operational parameters. The formal representations of the linear temporal logic specifications (LTLSEPCs) in nuXmv were derived by decomposition from the learning specifications in English, which were at a higher level of abstraction. Ten properties were verified with a total of 54 LTLSEPCs for the three sensors being used for the entire verification process. For each of the properties, counter-properties were also formulated to verify the completeness and

record any anomalies or edge cases for safety assessment. The learning performed by the LEIAS was verified by first generating the verification queries written in English text shown in Table 1.

7 | Results and Discussion

The LEIAS was formally verified within the AHMIAS framework, after an iterative interplay between formal verification and simulation, which identified errors in the IAS model, as well as translation errors. In the course of experimentation and translator updates to better align the formal verification with the learning specifications, there were challenges that needed to be addressed due to the limitations of nuXmv. For example, nuXmv constructs cannot handle tree-like structures and one of the instances of our implementation only had a single variable for current-value and old-value for the sensor errors associated with three types of errors. In Soar, due to the inherent tree structure, it corresponds to three current values, one for each of the errors, therefore retaining that information. This was solved by flattening the hierarchical data structure present in Soar into three separate error variables (*state_gps-error-info*, *state_imu-error-info*, *state_lidar-error-info*) to match nuXmv constructs. Due to the abovementioned change, each rule corresponding to the sensor error had to be triplicated. To make it modular and simpler, from a software engineering standpoint, macros were introduced. Macros efficiently compressed the Soar code by reducing the need for 33 rules to 20 rules. The execution of macros required the JSOAR debugger output before translation in nuXmv, as the macros were fully realized through the debugger.

8 | Conclusion and Future Work

In this research effort, we demonstrated the integration of learning in the AHMIAS framework. This was accomplished by identifying requirements for the HMI for a learning system and the architecture for the human-machine team. Finally, the simulation and verification of the learning-enabled IAS were performed to check the correctness of the IAS. Our approach showed how LEIAS can be designed with cognitive modeling, translated to a formal environment, and validated and verified through simulation and formal verification. Future work will involve investigating the life cycle management of LEIAS. Additionally, runtime assurance methods will be evaluated to perform assurance checks during learning by the LEIAS.

Acknowledgments

This work was funded by award 80NSSC20M0005 to the NASA Langley Research Center from the NASA Shared Services Center. The authors would also like to thank Paul Miner of NASA LaRC for his feedback on the learning function and its safety verification.

Data Availability Statement

The data that support the findings of this study are openly available in Github - AHMIAS at <https://github.com/loonwerks/AHMIAS>.

Endnotes

¹ <https://github.com/loonwerks/AHMIAS>

References

1. G. A. Cardona, C. Bravo, W. Quesada, et al., "Autonomous Navigation for Exploration of Unknown Environments and Collision Avoidance in Mobile Robots Using Reinforcement Learning," in *2019 SoutheastCon* (IEEE, 2019), 1–7.
2. N. Yan, S. Huang, and C. Kong, "Reinforcement Learning-Based Autonomous Navigation and Obstacle Avoidance for Uavs Under Partially Observable Conditions," *Mathematical Problems in Engineering* 2021 (2021): 1–13.
3. R. Rothfeld, A. Straubinger, M. Fu, C. Al Haddad, and C. Antoniou, "Urban Air Mobility," in *Demand for Emerging Transportation Systems* (Elsevier, 2020), 267–284.
4. A. Straubinger, R. Rothfeld, M. Shamiyeh, K.-D. Büchter, J. Kaiser, and K. O. Plötner, "An Overview of Current Research and Developments in Urban air Mobility—Setting the Scene for UAM Introduction," *Journal of Air Transport Management* 87 (2020): 101852.
5. O. A. Yakimenko and N. J. Slegers, "Using Direct Methods for Terminal Guidance of Autonomous Aerial Delivery Systems," in *2009 European Control Conference (ECC)* (IEEE, 2009), 2372–2377.
6. J. Saunders, S. Saeedi, and W. Li, "Autonomous Aerial Delivery Vehicles, a Survey of Techniques on how Aerial Package Delivery is Achieved," *arXiv preprint arXiv:2110.02429*, 2021.
7. A. Folkers, M. Rick, and C. Büskens, "Controlling an Autonomous Vehicle With Deep Reinforcement Learning," in *2019 IEEE Intelligent Vehicles Symposium (IV)* (IEEE, 2019), 2025–2031.
8. N. H. Campbell, M. J. Acheson, and I. M. Gregory, "Dynamic Vehicle Assessment for Intelligent Contingency Management of Urban Air Mobility Vehicles," *AIAA Scitech 2021 Forum* (ARC, 2021), <https://doi.org/10.2514/6.2021-1001>.
9. I. M. Gregory, N. H. Campbell, N. A. Neogi, et al., "Intelligent Contingency Management for Urban air Mobility," in *Dynamic Data Driven Applications Systems. DDDAS 2020. Lecture Notes in Computer Science* (ARC, 2021), https://doi.org/10.1007/978-3-030-61725-7_5.
10. D. P. Thipphavong, R. Apaza, B. Barmore, et al., "Urban air Mobility Airspace Integration Concepts and Considerations," in *2018 Aviation Technology, Integration, and Operations Conference* (ARC, 2018), 3676, <https://doi.org/10.2514/6.2018-3676>.
11. G. Gosian and K. Cohen, "Sparse Requirements Systems Engineering and Implications for Assured Autonomy," *Journal of Defense Management* 11 (2021): 17.
12. A. E. Goodloe, "Assuring Safety-Critical Machine Learning Enabled Systems: Challenges and Promise," in *2022 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)* (IEEE, 2022), 326–332.
13. E. E. Alves, D. Bhatt, B. Hall, K. Driscoll, A. Murugesan, and J. Rushby, "Considerations in Assuring Safety of Increasingly Autonomous Systems," (No. NASA/CR-2018-220080) (Technical report, 2018), <https://ntrs.nasa.gov/citations/20180006312>.
14. S. Bhattacharyya, J. Davis, A. Gupta, N. Narayan, and M. Matessa, "Assuring Increasingly Autonomous Systems in Human-Machine Teams: An Urban air Mobility Case Study," 348 (October 2021): 150–166, <https://doi.org/10.4204/eptcs.348.11>.
15. S. Bhattacharyya, N. Neogi, T. Eskridge, M. Carvalho, and M. Stafford, "Formal Assurance for Cooperative Intelligent Agents," in *Proceedings of NASA Formal Methods Symposium LNCS*, vol. 10811 (Springer, 2018), https://doi.org/10.1007/978-3-319-77935-5_2.
16. N. Narayan, P. Ganerwala, R. M. Jones, et al., "Assuring Learning-Enabled Increasingly Autonomous Systems," in *2023 IEEE International Systems Conference (SysCon)* (IEEE, 2023), 1–7, <https://doi.org/10.1109/SysCon53073.2023.10131227>.
17. N. Neogi, S. Bhattacharyya, D. Griessler, H. Kiran, and M. Carvalho, "Assuring Intelligent Systems: Contingency Management for uas," *IEEE*

- Transactions on Intelligent Transportation Systems 22, no. 9 (2021): 6028–6038, <https://doi.org/10.1109/TITS.2021.3076399>.
18. J. E. Laird, *The SOAR Cognitive Architecture* (MIT Press, 2012), ISBN 978-0262122962, <https://doi.org/10.7551/mitpress/7688.001.0001>.
19. G. Mani, B. Bhargava, P. Angin, M. Villarreal-Vasquez, D. Ulybyshev, and J. Kobes, “Machine Learning Models to Enhance the Science of Cognitive Autonomy,” in *2018 IEEE First International Conference on Artificial Intelligence and Knowledge Engineering (AIKE)* (IEEE, 2018), 46–53, <https://doi.org/10.1109/AIKE.2018.00015>.
20. V. Mnih, A. P. Badia, M. Mirza, et al., “Asynchronous Methods for Deep Reinforcement Learning,” in *International Conference on Machine Learning* (PMLR, 2016), 1928–1937.
21. P. Kulkarni, *Reinforcement and Systemic Machine Learning for Decision Making*, vol. 1 (John Wiley & Sons, 2012).
22. L. Hewing, K. P. Wabersich, M. Menner, and M. N. Zeilinger, “Learning-Based Model Predictive Control: Toward Safe Learning in Control,” *Annual Review of Control, Robotics, and Autonomous Systems* 3, no. 1 (2020): 269–296, <https://doi.org/10.1146/annurev-control-090419-075625>.
23. J. Kabzan, L. Hewing, A. Liniger, and M. N. Zeilinger, “Learning-Based Model Predictive Control for Autonomous Racing,” *IEEE Robotics and Automation Letters* 4, no. 4 (2019): 3363–3370.
24. J. Arroyo, C. Manna, F. Spiessens, and L. Helsen, “Reinforced Model Predictive Control (RL-MPC) for Building Energy Management,” *Applied Energy* 309 (2022): 118346.
25. R. Calinescu, D. Weyns, S. Gerasimou, M. U. Iftikhar, I. Habli, and T. Kelly, “Engineering Trustworthy Self-Adaptive Software With Dynamic Assurance Cases,” *IEEE Transactions on Software Engineering* 44, no. 11 (2017): 1039–1069.
26. B. H. C. Cheng, R. J. Clark, J. E. Fleck, M. A. Langford, and P. K. McKinley, “AC-ROS: Assurance Case Driven Adaptation for the Robot Operating System,” in *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems* (2020), 102–113.
27. D. Weyns and U. Iftikhar, “Model-Based Simulation at Runtime for Self-Adaptive Systems,” *Proceeding Models at Runtime, Würzburg* 2016 (2016): 1–9.
28. R. D. Avila and J. B. Clark, “Architecting Systems for Assured Autonomy,” in *2023 IEEE International Conference on Assured Autonomy (ICAA)* (IEEE, 2023), 91–96.
29. M. A. Langford, K. H. Chan, J. E. Fleck, P. K. McKinley, and B. H. C. Cheng, “Modalas: Model-Driven Assurance for Learning-Enabled Autonomous Systems,” in *2021 ACM/IEEE 24th International Conference on Model Driven Engineering Languages and Systems (MODELS)* (IEEE, 2021), 182–193.
30. D. Cofer, I. Amundson, R. Sattigeri, et al., “Run-Time Assurance for Learning-Enabled Systems,” in *Proceedings of the NASA Formal Methods: 12th International Symposium, NFM 2020, Moffett Field* (Springer-Verlag, 2020), 361–368, ISBN 978-3-030-55753-9, https://doi.org/10.1007/978-3-030-55754-6_21.
31. P. H. Feiler and D. P. Gluch, *Model-Based Engineering With AADL: An Introduction to the SAE Architecture Analysis & Design Language*, 1st ed. (Addison-Wesley Professional, 2012), ISBN 0321888944.
32. P. H. Feiler, B. A. Lewis, and S. Vestal, “The SAE Architecture Analysis & Design Language (AADL): A Standard for Engineering Performance Critical Systems,” in *IEEE International Symposium on Computer Aided Control System Design* (IEEE, 2006), 1206–1211.
33. M. W. Whalen, A. Gacek, D. Cofer, A. Murugesan, M. P. E. Heimdahl, and S. Rayadurgam, “Your ‘What’ Is My ‘How’: Iteration and Hierarchy in System Design,” *IEEE Software* 30, no. 2 (2013): 54–60, <https://doi.org/10.1109/MS.2012.173>.
34. H. Rueß, “Systems Challenges for Trustworthy Embodied Systems,” *arXiv preprint arXiv:2201.03413*, 2022, <https://arxiv.org/abs/2201.03413>.
35. C. Ellis, M. Wigness, and L. Fiondella, “A Mapping of Assurance Techniques for Learning Enabled Autonomous Systems to the Systems Engineering Lifecycle,” in *2022 IEEE International Conference on Assured Autonomy (ICAA)* (IEEE, 2022), 28–35.
36. G. Giray, “A Software Engineering Perspective on Engineering Machine Learning Systems: State of the art and Challenges,” *Journal of Systems and Software* 180 (2021): 111031.
37. J. Sprockhoff, B. Lukic, V. Janson, et al., “Model-Based Systems Engineering for AI-Based Systems,” in *AIAA SCITECH 2023 Forum* (2023), 2587.
38. I. G. Lopez-Francos, S. C. Mitchell, R. Lipkis, P. G. Vlastos, S. Mbaye, and S. I. Infeld, “A Model-Based Systems Engineering Approach for Developing an Autonomous Rover Testbed,” in *AIAA SCITECH 2023 Forum* (ARC, 2023), 1894.
39. N. Rajabli, F. Flammini, R. Nardone, and V. Vittorini, “Software Verification and Validation of Safe Autonomous Cars: A Systematic Literature Review,” *IEEE Access* 9 (2020): 4797–4819.
40. A. Gacek, J. Backes, M. Whalen, L. Wagner, and E. Ghassabani, “The Jkind Model Checker,” in *Computer Aided Verification* (2018).
41. L. De Moura and N. Björner, “Z3: An Efficient SMT Solver,” in *Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems, TACAS’08/ETAPS’08* (Springer-Verlag, 2008), 337–340, ISBN 3-540-78799-2, 978-3-540-78799-0.
42. D. Stewart, J. J. Liu, M. Heimdahl, D. Cofer, and M. Peterson, *Safety Annex for the Architecture Analysis and Design Language* (University of Minnesota, 2018).
43. R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction* (MIT Press, 2018).
44. S. Nason and J. Laird, “Soar-RL: Integrating Reinforcement Learning With Soar,” *Cognitive Systems Research* 6 (2005): 51–59, <https://doi.org/10.1016/j.cogsys.2004.09.006>.
45. K. Asadi and M. L. Littman, “An Alternative Softmax Operator for Reinforcement Learning,” in *Proceedings of the 34th International Conference on Machine Learning*, ed. D. Precup and Y. W. Teh, *Proceedings of Machine Learning Research*, vol. 70 (PMLR, August 2017), 243–252, <https://proceedings.mlr.press/v70/asadi17a.html>.
46. A. G. Nikolaev and S. H. Jacobson, “Simulated Annealing,” *Handbook of Metaheuristics* (Springer, 2010), 1–39.
47. R. Cavada, A. Cimatti, M. Dorigatti, et al., “The nuXmv Symbolic Model Checker,” in *CAV* (Springer, 2014), 334–342.
48. T. Parr, *The Definitive Antlr 4 Reference* (The Pragmatic Bookshelf, 2014).